

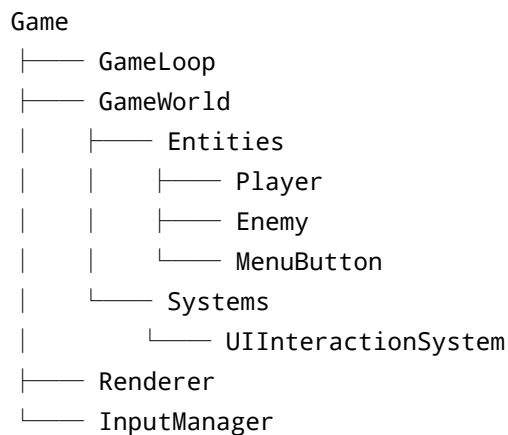
Sto creando un gioco in typescript. È strutturato con entities, gameworld, gameloop, e classi di rendering, tutte separate. Devo creare una classe che gestisce l'evento onhover del mouse sopra un canvas, e quando il puntatore è sopra una scritta questa si ingrandisce. Quando invece l'utente clicca sulla scritta inizia il gioco. Come devo strutturare la classe e come la inserisco in tutto il resto?

Mostra di più ▾



In una struttura con **GameLoop**, **GameWorld**, **Entities** e **Renderer** separati, conviene trattare l'interazione del mouse come un sistema dedicato, non come responsabilità del renderer.

Una possibile architettura è:



1. Crea un InputManager

Questa classe si occupa solo di leggere mouse e tastiera.

TypeScript

```
export class InputManager {
  public mouseX = 0;
  public mouseY = 0;
  public mouseClicked = false;

  constructor(private canvas: HTMLCanvasElement) {
    canvas.addEventListener("mousemove", this.onMouseMove);
    canvas.addEventListener("click", this.onClick);
  }

  private onMouseMove = (e: MouseEvent) => {
    const rect = this.canvas.getBoundingClientRect();

    this.mouseX = e.clientX - rect.left;
    this.mouseY = e.clientY - rect.top;
  };

  private onClick = () => {
    this.mouseClicked = true;
  };

  public reset() {
    this.mouseClicked = false;
  }
}
```

2. Crea un'entità UI

Ad esempio una scritta "START GAME".

TypeScript

```
export class MenuButton {
  constructor(
    public text: string,
    public x: number,
    public y: number,
    public width: number,
    public height: number
  ) {}

  public hovered = false;

  contains(mx: number, my: number): boolean {
    return (
      mx >= this.x &&
      mx <= this.x + this.width &&
      my >= this.y &&
      my <= this.y + this.height
    );
  }
}
```

3. Crea un sistema di interazione

Questa classe controlla hover e click.

TypeScript

```
export class UIInteractionSystem {
  constructor(
    private input: InputManager
  ) {}

  update(button: MenuButton, onClick: () => void) {

    button.hovered = button.contains(
      this.input.mouseX,
      this.input.mouseY
    );

    if (button.hovered && this.input.mouseClicked) {
      onClick();
    }
  }
}
```

4. Rendering separato

Il renderer legge solo lo stato.

TypeScript

```
export class MenuRenderer {

    render(
        ctx: CanvasRenderingContext2D,
        button: MenuButton
    ) {

        ctx.textAlign = "center";

        if (button.hovered) {
            ctx.font = "60px Arial";
        } else {
            ctx.font = "50px Arial";
        }

        ctx.fillStyle = "white";
        ctx.fillText(
            button.text,
            button.x + button.width / 2,
            button.y + button.height / 2
        );
    }
}
```

5. Nel GameLoop

Ogni frame:

TypeScript

```
gameLoop() {  
  
    uiSystem.update(startButton, () => {  
        gameState = GameState.PLAYING;  
    });  
  
    renderer.render();  
  
    input.reset();  
  
    requestAnimationFrame(() => this.gameLoop());  
}
```

Alternativa più scalabile: componente Clickable

Se prevedi più pulsanti (Start, Options, Exit, ecc.) conviene creare un'interfaccia:

TypeScript

```
export interface Clickable {  
    contains(x: number, y: number): boolean;  
    onClick(): void;  
    hovered: boolean;  
}
```

Il sistema UI scorre tutti gli elementi cliccabili:

TypeScript

```
for (const element of clickableElements) {  
  
    element.hovered =  
        element.contains(input.mouseX, input.mouseY);  
  
    if (element.hovered && input.mouseClicked) {  
        element.onClick();  
    }  
}
```

In questo modo il sistema non sa nulla di "Start Game", "Options" o altro: gestisce semplicemente elementi interattivi.

Per un progetto TypeScript ben organizzato, la divisione più pulita è:

- **InputManager** → raccoglie gli eventi del browser.
- **UIInteractionSystem** → decide hover e click.
- **MenuButton/UI entities** → contengono lo stato.
- **Renderer** → disegna in base allo stato.
- **GameLoop** → coordina tutto.

Così mantieni una separazione netta tra input, logica e rendering, che diventa molto importante quando il menu cresce o aggiungi altre schermate.

